

Loss Functions in Practice

Learning Objectives

By the end of this lecture, you should be able to:

- Explain what a loss function is and why it matters in training.
- Connect common losses to maximum likelihood via the negative log-likelihood (NLL).
- Choose appropriate losses for regression and classification.
- Derive (or recall) key gradients/subgradients for common losses.
- Handle practical issues: numerical stability, imbalance, and regularization.
- Work through small numeric examples and verify computations.

1 What Is a Loss Function?

Given data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and a model f_θ with parameters θ , a *loss function* $\ell(\hat{y}, y)$ maps prediction $\hat{y} = f_\theta(\mathbf{x})$ and true label y to a nonnegative number that quantifies *error*. We typically minimize the empirical risk:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(\mathbf{x}_i), y_i) \quad (+ \text{ regularization}).$$

Important: Loss vs. Metric. Training uses a differentiable *surrogate* loss (e.g., cross-entropy) even if our evaluation metric is non-differentiable (e.g., accuracy, F1, AUC). Always monitor the metric you care about on a validation set.

2 A Unifying View: Negative Log-Likelihood (NLL)

If your model specifies a conditional distribution $p_\theta(y \mid \mathbf{x})$, the natural choice is to minimize the *negative log-likelihood*

$$\ell_{\text{NLL}}(\theta; \mathbf{x}, y) = -\log p_\theta(y \mid \mathbf{x}).$$

Many standard losses appear as NLL under simple noise models:

Task	Likelihood $p_\theta(y \mid \mathbf{x})$	Loss (up to constants)
Regression	$y = f_\theta(\mathbf{x}) + \varepsilon, \varepsilon \sim \mathcal{N}(0, \sigma^2)$	$\frac{1}{2\sigma^2} \ f_\theta(\mathbf{x}) - y\ _2^2$ (MSE)
Binary class.	Bernoulli($\hat{p}$), $\hat{p} = \sigma(z)$	Binary cross-entropy (logistic loss)
Multiclass	Categorical(softmax($\mathbf{z}$))	Softmax cross-entropy

3 Regression Losses

Let $\hat{y} = f_{\theta}(\mathbf{x}) \in \mathbb{R}$.

Mean Squared Error (MSE)

$$\ell_{\text{MSE}}(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2.$$

Gradient: $\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y$.

When to use: Gaussian noise, smooth targets; sensitive to outliers.

Mean Absolute Error (MAE)

$$\ell_{\text{MAE}}(\hat{y}, y) = |\hat{y} - y|.$$

Subgradient: $\frac{\partial \ell}{\partial \hat{y}} \in \begin{cases} \{+1\}, & \hat{y} > y \\ [-1, +1], & \hat{y} = y \\ \{-1\}, & \hat{y} < y \end{cases}$

When to use: robust to outliers; targets with heavy tails.

Huber Loss (smoothly blends MSE/MAE)

For threshold $\delta > 0$,

$$\ell_{\text{Huber}}(\hat{y}, y) = \begin{cases} \frac{1}{2}(\hat{y} - y)^2, & |\hat{y} - y| \leq \delta, \\ \delta |\hat{y} - y| - \frac{1}{2}\delta^2, & \text{otherwise.} \end{cases}$$

Derivative: $\frac{\partial \ell}{\partial \hat{y}} = \begin{cases} \hat{y} - y, & |\hat{y} - y| \leq \delta, \\ \delta \text{sign}(\hat{y} - y), & \text{otherwise.} \end{cases}$

Numeric example (Huber). $y = 3.0$, $\hat{y} = 4.2$, $\delta = 1.0$. Then $|\hat{y} - y| = 1.2 > \delta$, so $\ell = \delta |\hat{y} - y| - \frac{1}{2}\delta^2 = 1.0 \cdot 1.2 - 0.5 = 0.7$, and $\frac{\partial \ell}{\partial \hat{y}} = \delta \text{sign}(1.2) = +1.0$.

4 Classification Losses

Let z denote a scalar logit (binary) or $\mathbf{z} \in \mathbb{R}^K$ logits (multiclass).

Binary Cross-Entropy (Logistic Loss)

For $y \in \{0, 1\}$ and logit $z \in \mathbb{R}$, $\hat{p} = \sigma(z) = 1/(1 + e^{-z})$:

$$\ell_{\text{BCE}}(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

Stable “with logits” form:

$$\ell_{\text{BCE-logits}}(z, y) = \max(z, 0) - z y + \log(1 + e^{-|z|}),$$

which avoids computing $\sigma(z)$ explicitly and is numerically stable.

Gradient wrt logit z :

$$\frac{\partial \ell}{\partial z} = \sigma(z) - y.$$

(Derivation: chain rule with $\hat{p} = \sigma(z)$ and $\partial \hat{p} / \partial z = \hat{p}(1 - \hat{p})$.)

Numeric example (BCE). $z = 1.2 \Rightarrow \hat{p} \approx 0.7685$. If $y = 1$, $\ell \approx -\log(0.7685) \approx 0.263$. If $y = 0$, $\ell \approx -\log(1 - 0.7685) \approx 1.466$.

Softmax Cross-Entropy (Multiclass)

For logits $\mathbf{z} \in \mathbb{R}^K$, softmax $p_k = \exp(z_k) / \sum_j \exp(z_j)$, true class $y \in \{1, \dots, K\}$:

$$\ell_{\text{CE}}(\mathbf{z}, y) = -\log p_y = -\left(z_y - \log \sum_{k=1}^K e^{z_k}\right).$$

Gradient wrt logits:

$$\frac{\partial \ell}{\partial z_k} = p_k - \mathbb{1}\{k = y\}.$$

Stability: implement with the *log-sum-exp* trick: $\log \sum_k e^{z_k} = \alpha + \log \sum_k e^{z_k - \alpha}$ with $\alpha = \max_k z_k$.

Numeric example (softmax CE). $\mathbf{z} = (2.0, 1.0, 0.0)$, $y = 1$ (1-indexed). Then $\sum e^{z_k} = e^2 + e^1 + e^0 \approx 7.389 + 2.718 + 1 = 11.107$, $p_1 \approx 7.389/11.107 \approx 0.665$, so $\ell \approx -\log(0.665) \approx 0.408$.

Label Smoothing

Replace one-hot target $\mathbf{e}_y$ by $\tilde{\mathbf{y}} = (1 - \varepsilon)\mathbf{e}_y + \frac{\varepsilon}{K}\mathbf{1}$, $0 < \varepsilon < 1$, and compute CE with $\tilde{\mathbf{y}}$. *Benefits:* reduces overconfidence and can improve calibration.

5 Margins, Ranking, and Embeddings

Hinge Loss (SVM)

For $y \in \{-1, +1\}$ and score s ,

$$\ell_{\text{hinge}}(s, y) = \max\{0, 1 - ys\}.$$

Zero loss if $ys \geq 1$; otherwise linear penalty.

Triplet Loss (Representation Learning)

Given anchor a , positive p , negative n , margin $m > 0$,

$$\ell_{\text{triplet}} = \max\{0, d(f_{\theta}(a), f_{\theta}(p)) - d(f_{\theta}(a), f_{\theta}(n)) + m\},$$

with d a distance (e.g., Euclidean). Encourages a closer to p than n by margin m .

6 Imbalanced Data and Hard Examples

Class Weighting

Weight each sample inside BCE/CE by w_y to counter class imbalance. Common heuristic: $w_c \propto 1/\text{freq}(c)$.

Focal Loss (Binary)

Focus on hard/misclassified examples, $\gamma \geq 0$, class weight $\alpha \in [0, 1]$:

$$\ell_{\text{focal}}(z, y) = -\alpha y (1 - \hat{p})^\gamma \log \hat{p} - (1 - \alpha) (1 - y) \hat{p}^\gamma \log(1 - \hat{p}), \quad \hat{p} = \sigma(z).$$

When $\gamma = 0$, this reduces to weighted BCE.

Numeric example (focal). $z = 2.0 \Rightarrow \hat{p} \approx 0.881$, $y = 1$, $\alpha = 0.25$, $\gamma = 2$: $\ell \approx -0.25 (1 - 0.881)^2 \log(0.881) \approx -0.25 \cdot 0.014 \cdot (-0.127) \approx 0.00044$ (very easy sample $\Rightarrow$ down-weighted).

7 Sequences and Structured Prediction

- **Token-level CE** summed over time steps; *mask* padded tokens and normalize by the number of valid tokens.
- **CTC loss** (Connectionist Temporal Classification): for speech/handwriting when frame-to-label alignment is unknown.

8 Calibration and Probabilistic Quality

Brier score (binary) measures accuracy and calibration:

$$\ell_{\text{Brier}} = (\hat{p} - y)^2.$$

Well-calibrated models produce probabilities that match observed frequencies. (Advanced: expected calibration error, reliability diagrams.)

9 Regularization: Adding Terms to the Loss

To control complexity and improve generalization,

$$L_{\text{total}}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell_i(\theta) + \lambda \Omega(\theta).$$

- **L2 (weight decay):** $\Omega(\theta) = \|\theta\|_2^2$ (Gaussian prior view).
- **L1:** $\Omega(\theta) = \|\theta\|_1$ promotes sparsity (Laplace prior view).
- **Decoupled weight decay (AdamW):** apply decay as a separate step for better behavior with adaptive optimizers.

10 Choosing a Loss: A Practical Checklist

- **Task type:** regression ($\rightarrow$ MSE/MAE/Huber) vs. classification ($\rightarrow$ BCE/CE).
- **Noise/outliers:** prefer MAE/Huber if heavy-tailed.
- **Imbalance:** add class weighting or focal loss.

- **Probabilities needed?** cross-entropy losses yield calibrated scores (with proper training); consider label smoothing.
- **Numerical stability:** use logits-based APIs (e.g., BCEWithLogits, CrossEntropy) and log-sum-exp.
- **Metric mismatch:** if optimizing F1/AUC, use surrogates and validate carefully.

11 Worked Mini-Examples (End-to-End)

Regression (MSE) with a Single Feature

Data: $(x, y) = (2.0, 5.0)$. Model $\hat{y} = wx + b$ with $(w, b) = (1.0, 0.0)$.

$$\hat{y} = 1 \cdot 2 + 0 = 2, \quad \ell = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(2 - 5)^2 = 4.5.$$

Gradients: $\partial\ell/\partial\hat{y} = \hat{y} - y = -3$, $\partial\ell/\partial w = (\hat{y} - y)x = -6$, $\partial\ell/\partial b = (\hat{y} - y) = -3$.

Binary Classification (BCE with logits)

Logit $z = -0.3 \Rightarrow \hat{p} = \sigma(-0.3) \approx 0.425$. If $y = 1$, $\ell = -\log(0.425) \approx 0.856$ and $\partial\ell/\partial z = \hat{p} - y \approx -0.575$.

Multiclass (Softmax CE)

Logits $\mathbf{z} = (0.5, 0.0, -0.5)$, $y = 2$. Softmax: $p_1 \approx 0.506$, $p_2 \approx 0.307$, $p_3 \approx 0.187$; loss $\ell = -\log p_2 \approx 1.179$. Gradients wrt logits: $p_k - \mathbb{1}\{k = y\}$.

12 Implementation Tips (Framework-Agnostic)

- Prefer built-in *logits* versions for stability (e.g., BCEWithLogits, CrossEntropy).
- For sequences, always **mask** padded positions and normalize properly.
- With class imbalance, verify weights sum/scale as intended (and monitor per-class metrics).
- For multi-task training, balance loss terms so that gradient magnitudes are comparable.

Exercises (Short)

1. Show that the gradient of softmax CE wrt logits is $p_k - \mathbb{1}\{k = y\}$. (*Hint:* differentiate $-\log p_y$.)
2. Derive the Huber derivative given in this lecture.
3. For BCE with logits, derive $\partial\ell/\partial z = \sigma(z) - y$ starting from $-y \log \hat{p} - (1 - y) \log(1 - \hat{p})$ and $\hat{p} = \sigma(z)$.
4. Numeric: with $\delta = 1$, $y = 0$, $\hat{y} \in \{-1.2, -0.3, 0.6\}$, compute Huber loss values.